



Python Programming (BCC-302)

(UNIT – 1)

Content

- Introduction
- Software Installation and Getting Started
- How to use Python Programming (Console and GUI)
- Python Variables
- Operator
- Blocks
- Data Types



Introduction

- Python is developed by **Guido Van Rossum**, in 1991.
- Python was named after a TV show, Monty Python's Flying Circus.
- Python is HLL, interpreted, interactive programming language
- Python supports OOPs Concept
- Python is case sensitive language
- Python is an embedded scripting language
- Major implementations are CPython, IronPython, JPython, PyPy
- Used for developing web applications



Why Python?

- Python is fast, efficient, portable
- Python rapidly iterate and invent
- Python has amazing libraries
- Python has less development time than other languages
- Largest year-on-year demand
- Better Jobs for freshers to experience person
- Use for image processing, data analysis, to creating games, web applications & many more.



Comparison with Other Languages

- Python supports the use of FUNCTIONS and CLASSES but does not force it
- For example, to print “*HELLO WORLD !*” program in Python

```
>>> print (“HELLO WORLD !”)
```

- C++ requires this program as function wrapping and preceded by a preprocessor directive

```
# include <iostream.h>
```

```
void main()
```

```
{
```

```
    cout <<“HELLO WORLD !” ;
```

```
}
```



Comparison with Other Languages

- In Java, situation is even worse, as all code must be inside of a class

```
public class c1
{
    public static void main (String [] args)
    {
        System.out.println("HELLO WORLD !");
    }
}
```



Comparison with Other Languages

- The same condition repeats with C# also

```
class c1
```

```
{
```

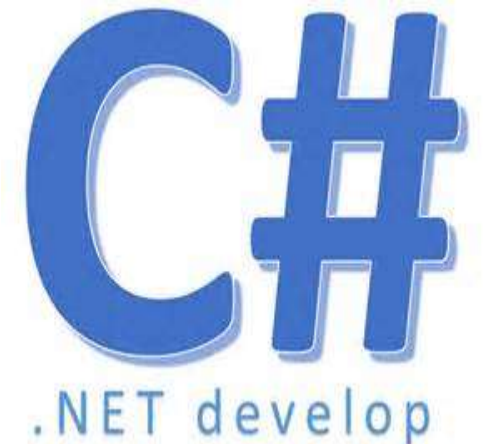
```
    public static void Main()
```

```
{
```

```
    System.Console.WriteLine ("HELLO WORLD !");
```

```
}
```

```
}
```



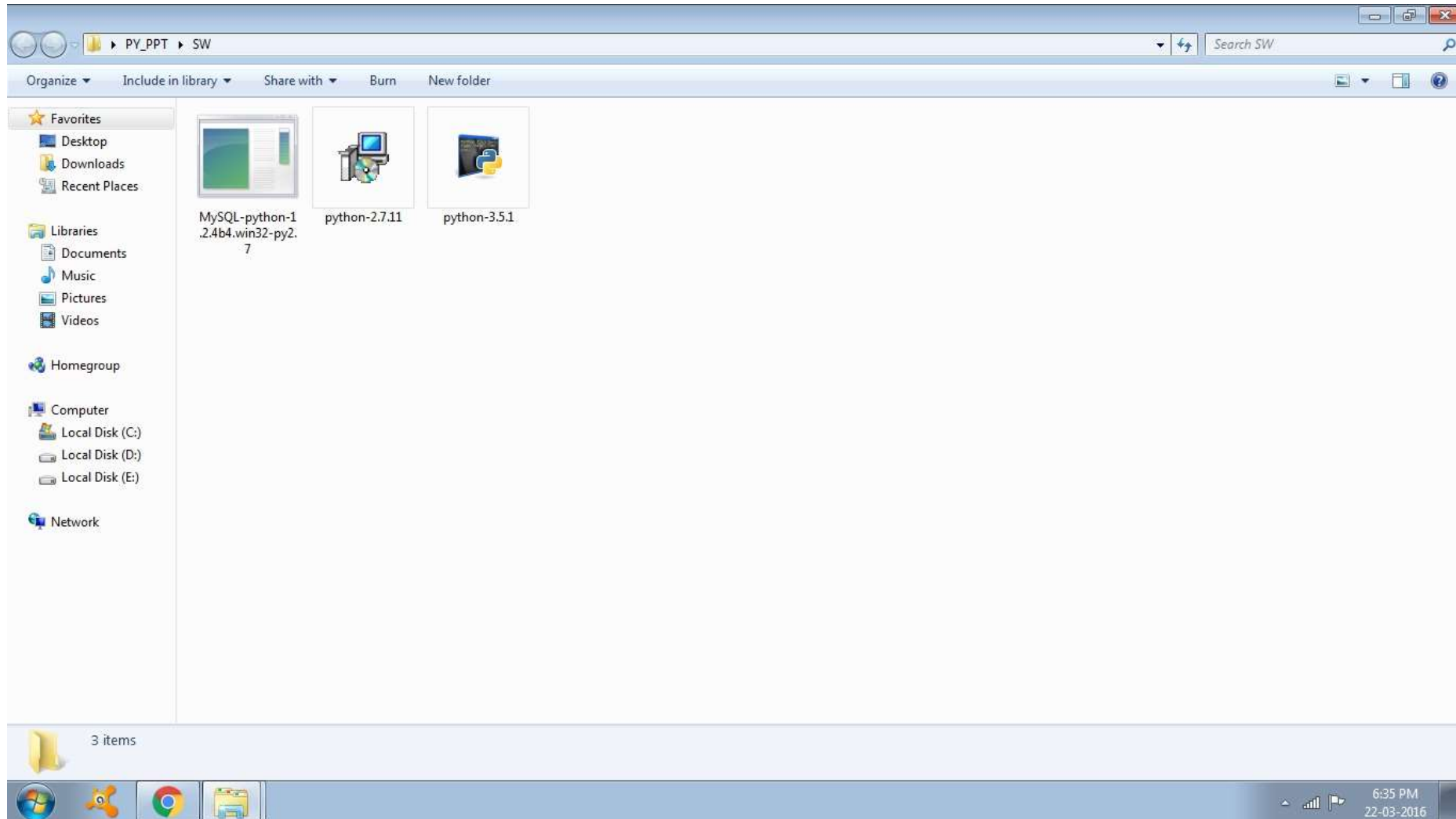
Comparison with Other Languages

- This situation slightly improves with PHP code

```
< ? php  
    echo "HELLO WORLD ! ";  
?>
```



Python Installation (Step 1)



Python Installation (Step 2)



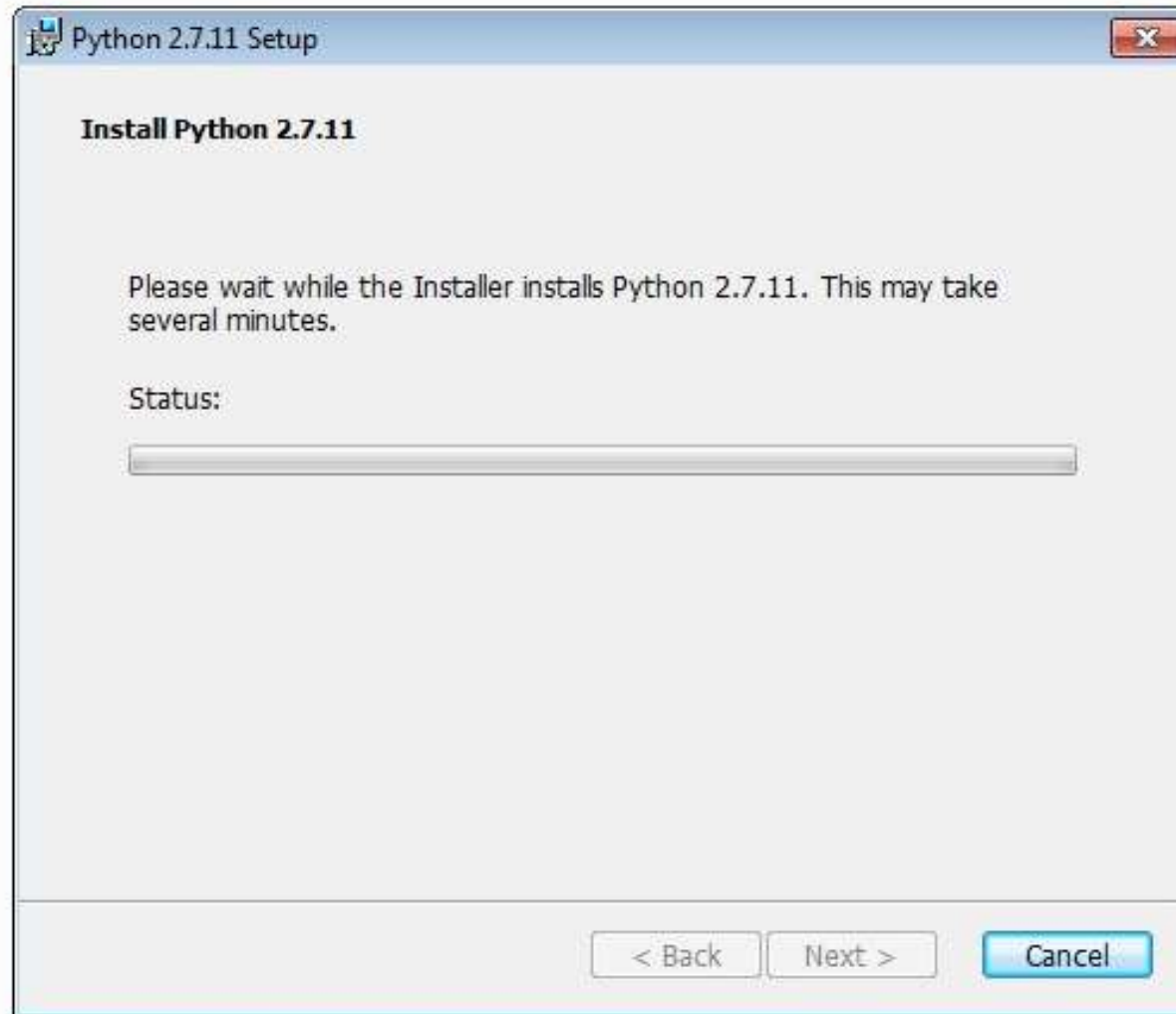
Python Installation (Step 3)



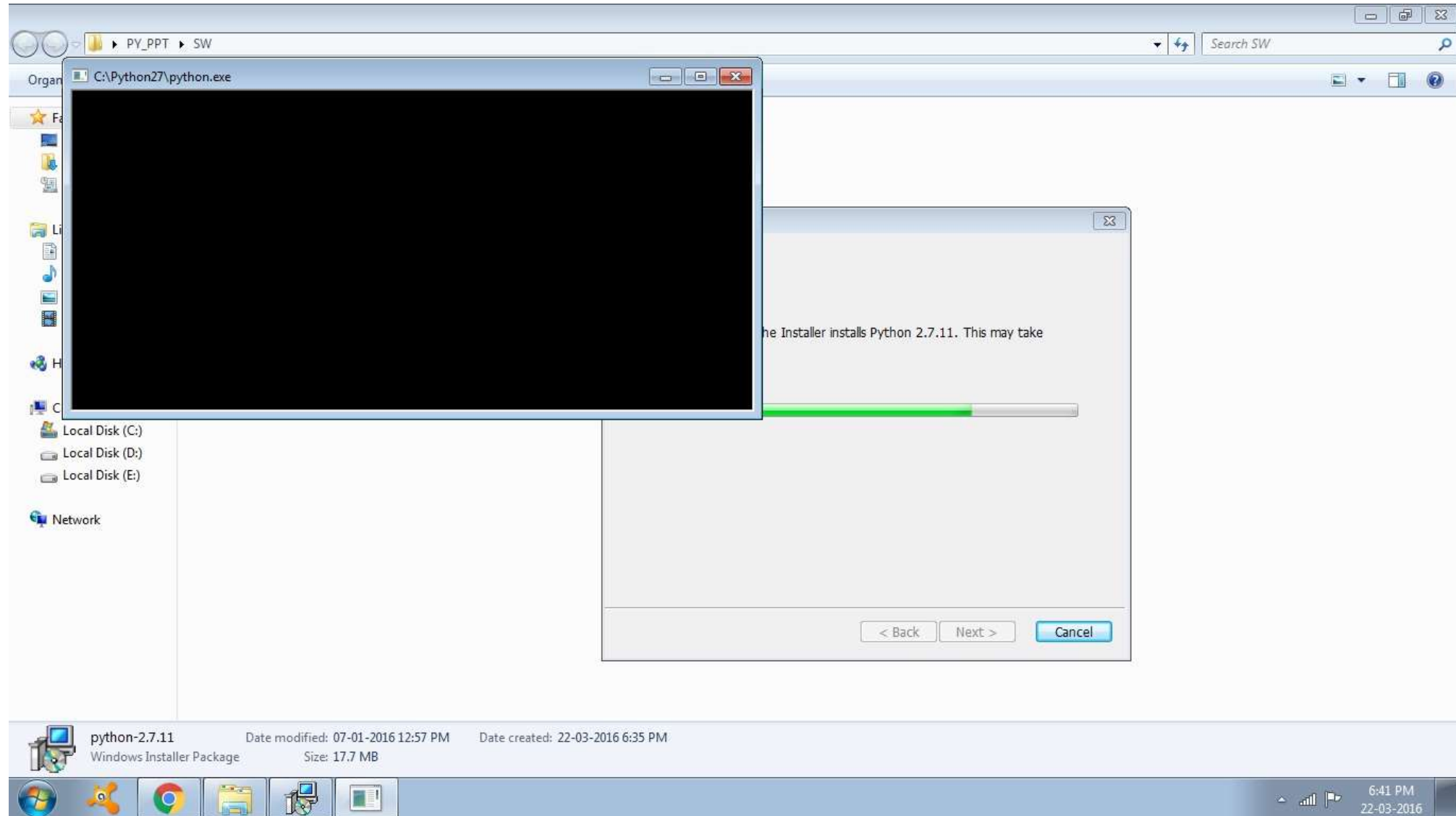
Python Installation (Step 4)



Python Installation (Step 5)



Python Installation (Step 6)



Python Installation (Step 7)



How to use Python

- **Interactive Mode Programming**

Invoking the interpreter without passing a script file.

Note: If we are using the older version of Python (Python 2.x), use of parenthesis as in **print** function is optional.

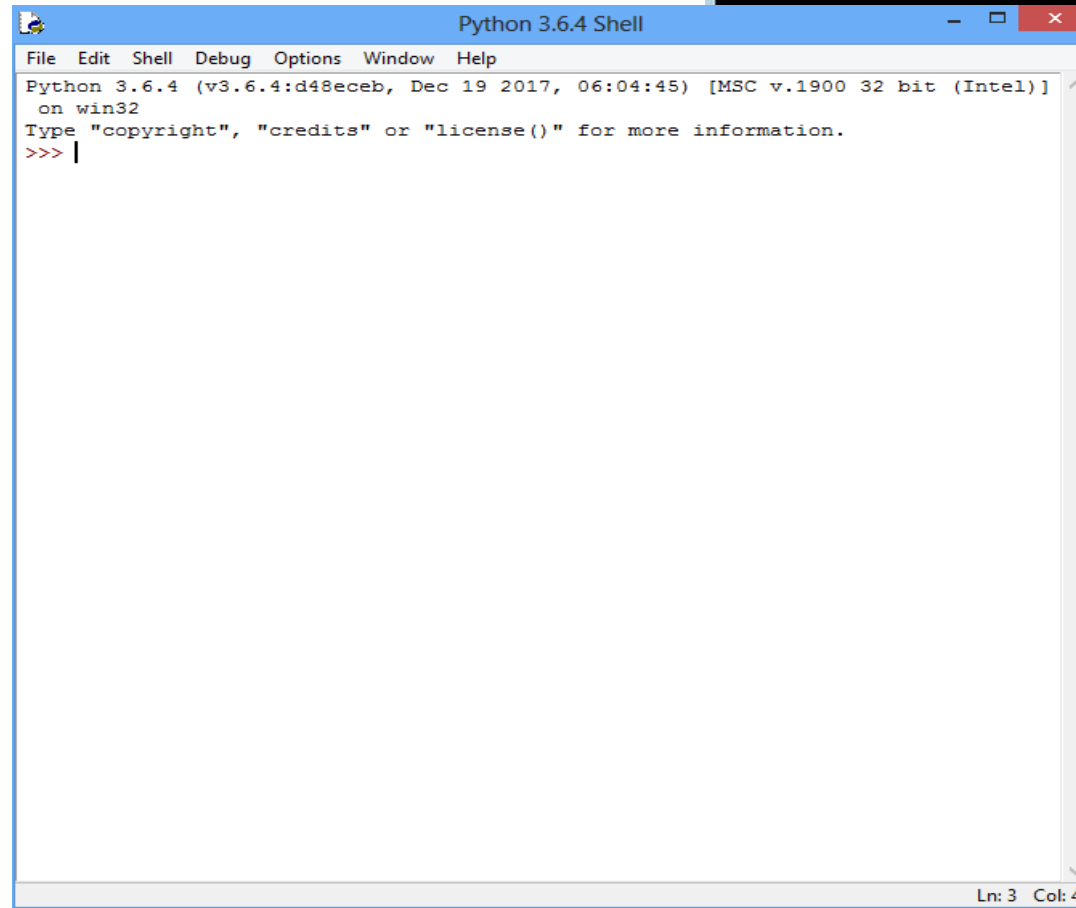
- **Script Mode Programming(GUI)**

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.



How to use Python (cont.)

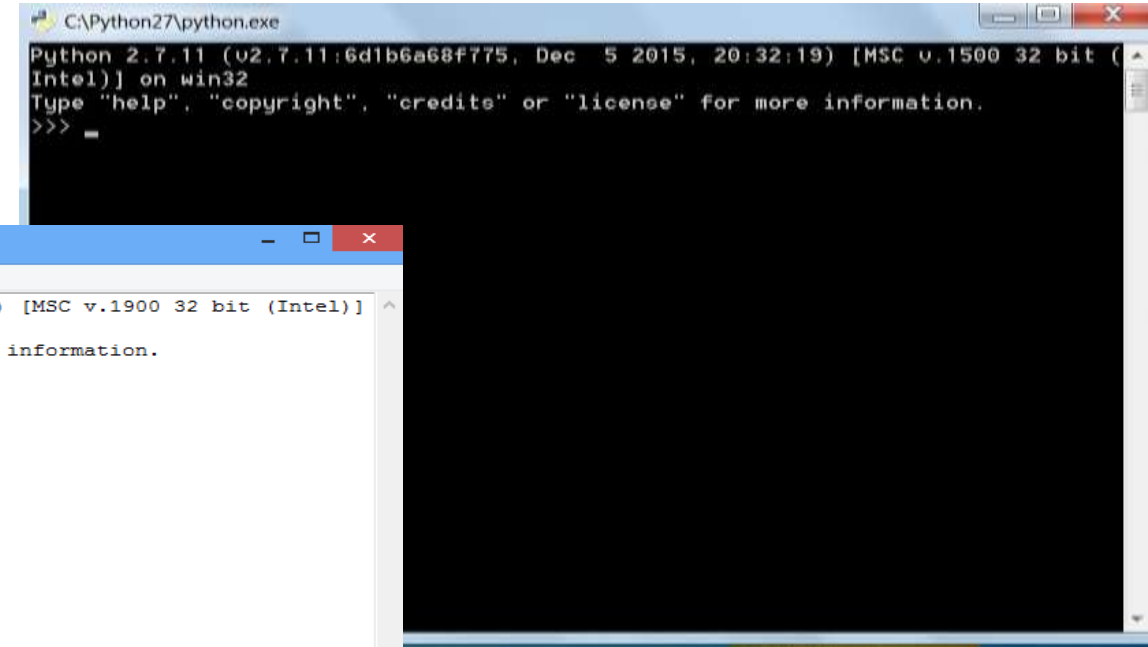
- Interactive Mode Programming



A screenshot of the Python 3.6.4 Shell window. The window has a blue title bar and a menu bar with options: File, Edit, Shell, Debug, Options, Window, and Help. The main text area shows the following text: "Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)] on win32", "Type 'copyright', 'credits' or 'license()' for more information.", and the interactive prompt ">>> |". The status bar at the bottom right indicates "Ln: 3 Col: 4".

```
Python 3.6.4 Shell
File Edit Shell Debug Options Window Help
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>> |
```

Ln: 3 Col: 4

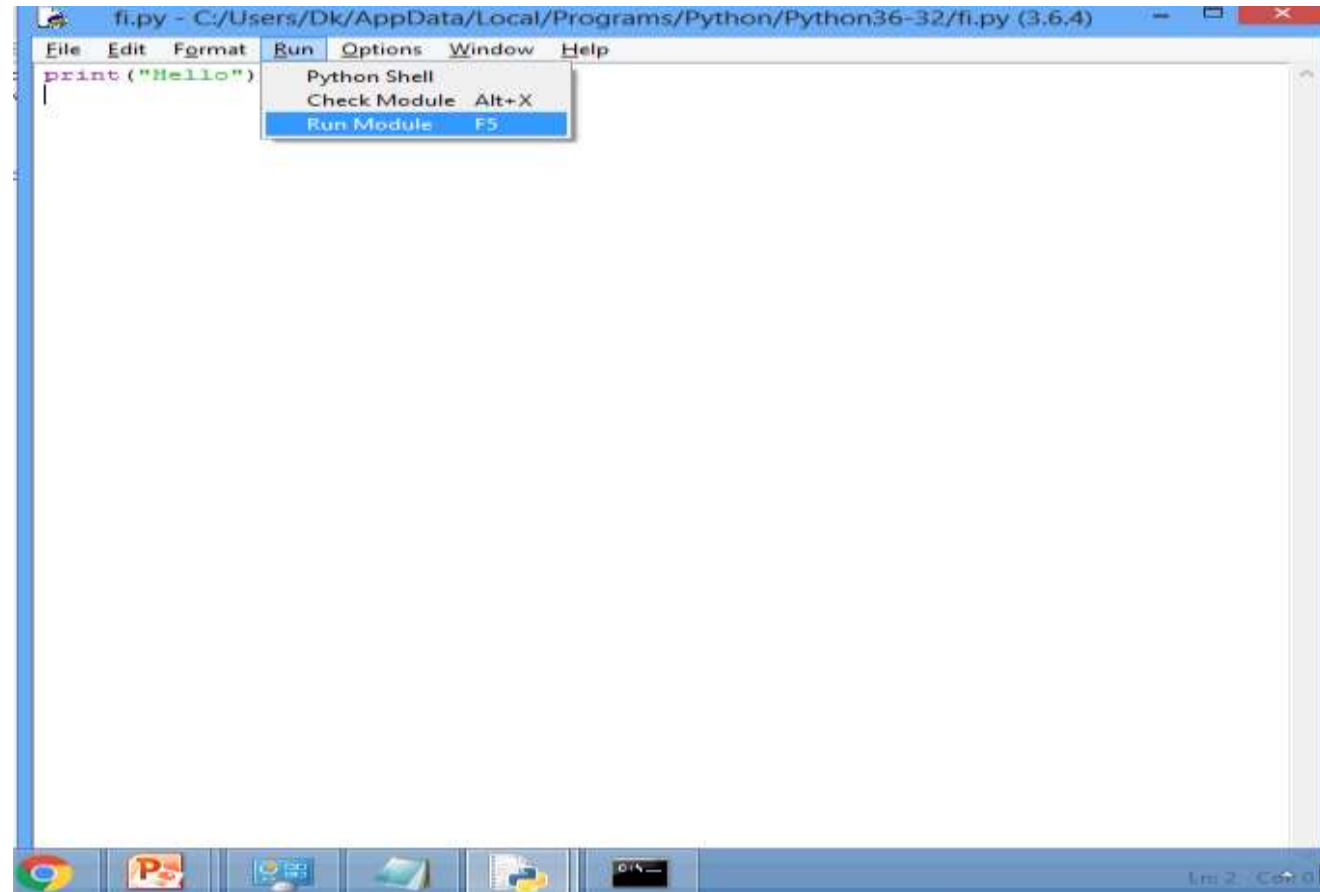


A screenshot of the Python 2.7.11 window. The window has a blue title bar and a menu bar. The main text area shows the following text: "Python 2.7.11 (v2.7.11:6d1b6a68f775, Dec 5 2015, 20:32:19) [MSC v.1500 32 bit (Intel)] on win32", "Type 'help', 'copyright', 'credits' or 'license' for more information.", and the interactive prompt ">>> |".

```
C:\Python27\python.exe
Python 2.7.11 (v2.7.11:6d1b6a68f775, Dec 5 2015, 20:32:19) [MSC v.1500 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

How to use Python (cont.)

- Script Mode Programming



Python Input-Output

- **print()-**

This is used to display messages or values of variables. It inserts a new line at the end, by default.

In Python 3, end=" " appends space instead of newline.

Syntax: print ("Message",VarList)

Note: In python 3 Use of parenthesis () with print function is mandatory(Optional In python 2).

- **Input()**

This is used to take input from user. By default it returns string type value.

Syntax: Var=input ("Message")



Variables/Identifiers

- A Python identifier is a name used to identify a variable, function, class, module or other object.
- An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow punctuation characters such as @, \$, and % within identifiers. Python is a case sensitive programming language.
- Are not declared, just assigned
- The variable is created the first time you assign it a value
- Are references to objects
- Type information is with the object, not the reference
- Everything in Python is an object



Python Reserved Keywords

And	Exec	Not
As	Finally	Or
Assert	For	Pass
Break	From	Print
Class	Global	Raise
Continue	If	Return
Def	Import	Try
Del	In	While
Elif	Is	With
Else	lambda	yield
except		



Operators

- Operator is symbol, used to perform operations in python.
- Types of Operator
 - Arithmetic Operators
 - Comparison (Relational) Operators
 - Assignment Operators
 - Logical Operators
 - Bitwise Operators
 - Membership Operators
 - Identity Operators



Operator (cont.)

- **Arithmetic Operators**

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 31$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -11$
* Multiplication	Multiplies values on either side of the operator	$a * b = 210$
/ Division	Divides left hand operand by right hand operand	$b / a = 2.1$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 1$
** Exponent	Performs exponential (power) calculation on operators	$a^{**}b = 10 \text{ to the power } 20$
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9//2 = 4$ and $9.0//2.0 = 4.0$, - $11//3 = -4$, $-11.0//3 = -4.0$



Operator (cont.)

- **Comparison Operators**

Operator	Description	Example
==	If the values of two operands are equal, then the condition becomes true.	(a == b) is not true.
!=	If values of two operands are not equal, then condition becomes true.	(a != b) is true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.	(a > b) is not true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.	(a < b) is true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.	(a >= b) is not true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.	(a <= b) is true.



Operator (cont.)

- **Assignment Operators**

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
<code>+=</code> Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
<code>-=</code> Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c - a$
<code>*=</code> Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	$c *= a$ is equivalent to $c = c * a$
<code>/=</code> Divide AND	It divides left operand with the right operand and assign the result to left operand	$c /= a$ is equivalent to $c = c / a$ $c /= a$ is equivalent to $c = c / a$
<code>%=</code> Modulus AND	It takes modulus using two operands and assign the result to left operand	$c \% a$ is equivalent to $c = c \% a$
<code>**=</code> Exponent AND	Performs exponential (power) calculation on operators and assign value to the left operand	$c ** a$ is equivalent to $c = c ** a$
<code>//=</code> Floor Division	It performs floor division on operators and assign value to the left operand	$c //= a$ is equivalent to $c = c // a$



Operator (cont.)

- **Bitwise Operators**

Operator	Description	Example
& Binary AND	Operator copies a bit, to the result, if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit, if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit, if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operand's value is moved left by the number of bits specified by the right operand.	a << = 240 (means 1111 0000)
>> Binary Right Shift	The left operand's value is moved right by the number of bits specified by the right operand.	a >> = 15 (means 0000 1111)



Operator (cont.)

- **Logical Operators**

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is False.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is True.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is True.



Operator (cont.)

- **Membership Operators**

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.



Operator (cont.)

- Identity Operators

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).



Operator Precedence

()	Parentheses
**	Exponent (R to L)
+x,-x,~x	Unary plus, Unary minus, Bitwise NOT
*,/,//,%+	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
!	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity. Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR
=+=, -=, =, /=	Assignment operator (R to Left)



Python Blocks

- Indentation is used to define code blocks in Python.
- Correct indentation is mandatory improper indentation causes error.
- Unlike other languages (C, Java), Python does not use {} brackets.
- Avoid mixing tabs and Spaces

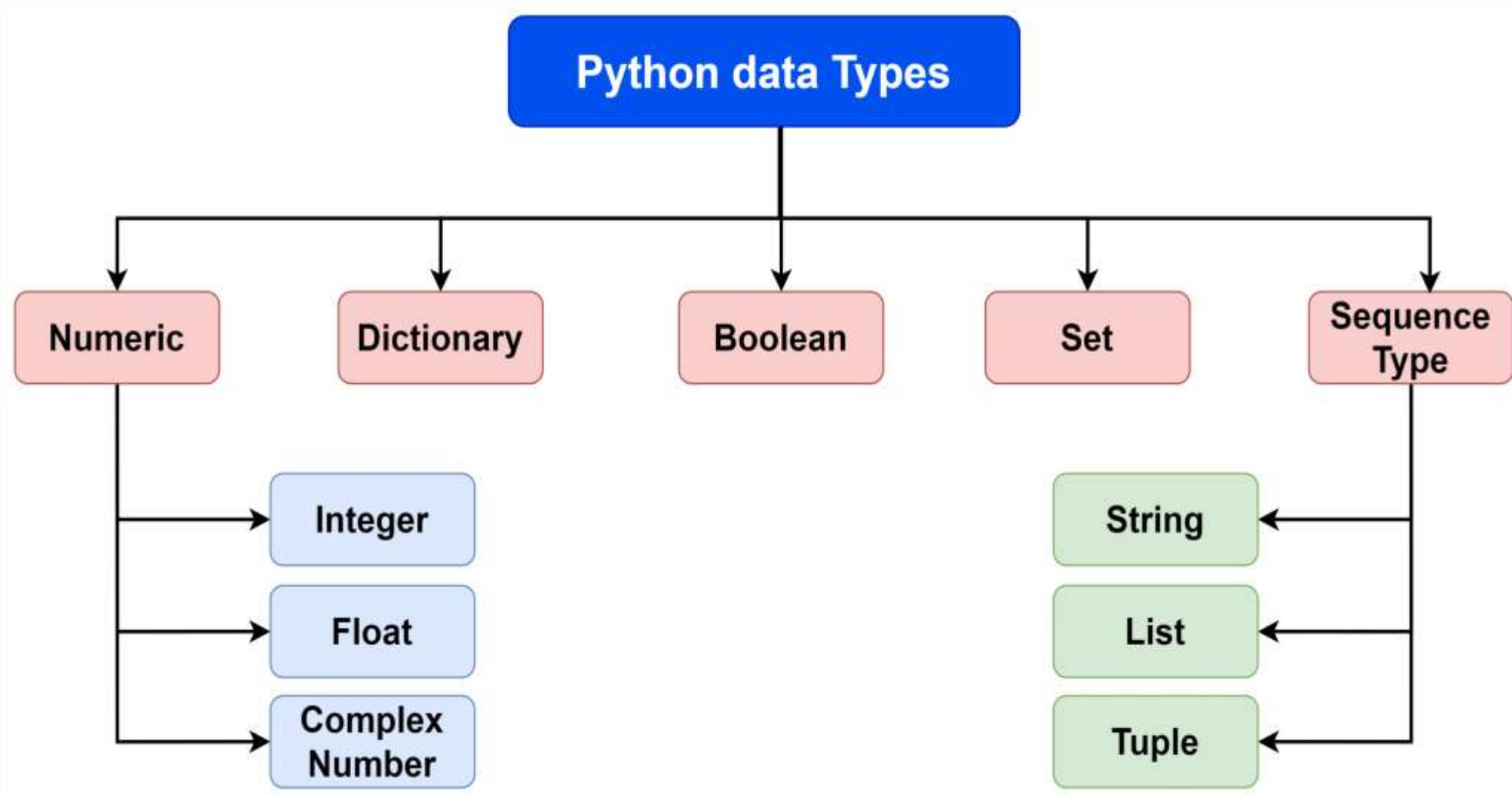
if True:

```
    print("This is inside the block") # Indented
```

```
print("This is outside the block") # Not indented
```



Data Types



Data Type Chart

	Ordered	Mutable	Duplicate	Use Case
List	Y	Y	Y	General purpose, maintain order
Tuple	Y	N	Y	Fixed data, safe from modification
Set	N	Y	N	Unique items, fast membership check
Dictionary	Y	Y	N	Key-value mappings, fast lookups

Type Casting

- Type casting is the process of converting data from one type (data type) to another.



- Type casting of two types:
 - **Implicit type casting**
Conversion from one type to another perform automatically.
 - **Explicit type casting**
Conversion from one type to another perform manually.
Syntax- `TargetType(Variable/Value);`



S.No.	Function & Description
1	int(x [,base]) Converts x to an integer. The base specifies the base if x is a string.
2	float(x) Converts x to a floating-point number.
3	complex(real [,imag]) Creates a complex number.
4	str(x) Converts object x to a string representation.
5	repr(x) Converts object x to an expression string.
6	eval(str) Evaluates a string and returns an object.
7	tuple(s) Converts s to a tuple.
8	list(s) Converts s to a list.
9	set(s) Converts s to a set.
10	dict(d) Creates a dictionary. d must be a sequence of (key,value) tuples.
11	frozenset(s) Converts s to a frozen set.
12	chr(x) Converts an integer to a character.
13	unichr(x) Converts an integer to a Unicode character.
14	ord(x) Converts a single character to its integer value.
15	hex(x) Converts an integer to a hexadecimal string.
16	oct(x) Converts an integer to an octal string.



Practice Questions

1. Declare a variable to store your name and print it.
2. Divide two numbers and print the result as float.
3. Use floor division on two numbers and print the output.
4. Declare a variable and assign None to it. Print it.
5. Swap values of two variables and print them.
6. Compare two numbers using all comparison operators.
7. Use exponent to raise a number to a power and print it.
8. Convert a float to integer and print the result.
9. Compute two complex numbers and print it.
10. Create a complex number and print the real and imaginary part.



Thank You

